Risk Register — Jeopardy Wagering Model

Last updated: 2026-10-13

Note (2026-10-13, release 2.1.0): risks #5, #12 and #14 updated and risk

15 added, after the private calibration of the simulated contestants to

modern-era play (ADR-0004). Calibrated values are not stated here, because the source data may not be republished; see `engine/behavior.py`.

Note (2026-10-13, release 2.0.0): risks #5, #8 and #12 updated and risk #14 added, after the move to simulated contestant behavior and the tiebreaker rule (ADR-0003).

Note (2026-07-23): risks #9 and #10 below document a two-layer defect in the Daily Double engine's statistics — the first fix (risk #9) was itself found wrong on a deeper review (risk #10) and re-fixed. Prior conclusions in this register are not rewritten when superseded — each row keeps its own history so the sequence of what changed and why stays legible. See the changelog and `docs/reviews/statistical-review-2026-07-23-v2.md`.

#	Risk	Likelihood	Impact	Mitigation	Status
1	Historical data licensing restricts what appears in shipped artifacts	High	Medium	Cite Tesauro et al. (2013) published statistics for anything shipped; restrict jwolle1 dataset to internal validation only. See ADR-0001.	Accepted, mitigated
2	Real player confidence is unobservable in historical data, limiting how prescriptive the model can be	High	High	Require a self-reported confidence input (e.g., 1–5 scale mapped to historical row-based accuracy) rather than inferring confidence purely from data	Open — design decision needed
3	Live in-game wagering assistance would violate actual contest rules	N/A (policy)	High	Scope the tool explicitly as pre-game study and post-game analysis only; state this in the README and white paper	Open — needs explicit statement in shipped docs
4	Pairwise heuristic decomposition for 3-player wagers is an approximation, not a proven joint optimum	Medium	Medium	Validate heuristic layer against the full N-player Monte Carlo simulator before treating it as authoritative	Resolved — Tier 1 (`fj_wager.py`) uses a single largest-threat formula, not pairwise decomposition; see section 3.2 of the research memo

#	Risk	Likelihood	Impact	Mitigation	Status
5	Tier 2 (`fj_bestresponse.py`) and the Daily Double engine (`dd_wager.py`, `simulate.py`) depend on an opponent-behavior model that is only partly calibrated. Since 2.1.0 (ADR-0004, `engine/behavior.py`) these values are CALIBRATED privately to modern-era play (Seasons 38-41, 2021-2025): Final Jeopardy accuracy by score rank and overall, the right/wrong correlation, the leader's cover share, Daily Double accuracy, the future-DD wager fraction by round, and the future-DD finder weights by score rank. Second place's wager mix in one of six regimes (B >= 3/4 A and B >= 2C) is PUBLISHED (Tesauro et al., 2013, 1990s-2012 games). These remain ASSUMED: second place's mix in the other five regimes, the split of the leader's bets, third place's flat shape, the regular-clue success rate (placeholder), and the finder's fixed wager fraction (real finders choose from the game state). The correlation conversion (Sheppard's formula) is exact only at even accuracy and approximate at modern accuracy. The finder weights use the returning champion's lectern as a proxy for the leader. The published B weights rest on groups of only a few hundred samples each.	High	Medium	Every value sits in one configuration block, marked CALIBRATED, PUBLISHED or ASSUMED, and is mirrored in the app's `BEHAVIOR` object. Hand-derived tests use explicit fixed models, so they stay valid when a value changes. Win percentages are presented as model outputs, not validated statistics. The calibrated values cannot be checked by readers against their source (ADR-0004).	Partly mitigated (2.1.0) — second place's mix and the regular-clue model remain to calibrate; no modern source for second place's mix exists

#	Risk	Likelihood	Impact	Mitigation	Status
6	Monte Carlo win probabilities in the Daily Double engine carry sampling noise, which can make a worse wager look better by chance at low trial counts	Medium	Low	Implemented common random numbers (paired random draws across candidate wagers within each trial) to remove most of this noise; see comment in `dd_wager.py::analyze`	Mitigated — raise `trials_per_wager` for tighter estimates if still needed
7	Regular-clue and future-Daily-Double dynamics in `simulate.py` ignore buzzer competition and wrong-answer pass-through	Medium	Medium	Documented as a named simplification in the module docstring; results from multi-clue-ahead or multi-DD-ahead rollouts should be treated as rough, not precise	Open — refine before trusting rollouts far from the last Daily Double
8	`app/index.html` is a hand-written JavaScript port of the Python engine, not generated from it -- the two can silently drift apart as either one changes. The two engines also use different PRNGs (Python: Mersenne Twister; JS: mulberry32), so their Monte Carlo Daily Double outputs are only distributionally comparable, never identical trial-for-trial.	Medium	High	`tests/js_parity.js` extracts the app's engine and checks it against the Python hand-derived values: Tier 1, Tier 2 (including the tiebreaker value of a tie bet and the correlated case), the simulated-behavior wager mixes, the inverse-normal and Bonferroni-band reference values, and the paired-SE <= independent-SE invariant. CI runs it on every push and pull request (since 1.4.0). The `BEHAVIOR` object must be edited by hand to match `engine/behavior.py`; the parity checks catch a mismatch in the tested cases only.	Mitigated — automated parity checks in CI; Monte Carlo outputs are checked for invariants only
9	Daily Double engine picked the "best" wager by comparing noisy Monte Carlo win-probability estimates with a fixed 1e-9 tolerance, treating them as exact. Reported 2026-07-23: a 2-point change in stated confidence (42% → 44%) swung the recommended wager from $350 to $4,200 for identical scores -- a real defect, not a data problem.	High (was live in shipped app)	High	First fix (2026-07-23, v1): standard-error-aware "plateau" selection. **Superseded by the v2 fix below — the v1 fix's SE formula was itself wrong (see risk #10). Kept here unrewritten for history.**	Resolved by v1, then found insufficient — see risk #10

#	Risk	Likelihood	Impact	Mitigation	Status
10	The v1 fix for risk #9 computed the wager-comparison standard error with the INDEPENDENT two-sample formula `sqrt(se_w^2 + se_best^2)`, but the simulator uses common random numbers (CRN), under which candidate wagers are strongly positively correlated. The independent formula therefore over-stated the comparison variance, producing a spuriously wide plateau and systematic UNDER-betting (it recommended ~$0 even where betting was clearly better; for `[4200,6000,3400]` @0.42 the independent rule recommended $100 where the CRN-correct rule recommended $4,200).	High (was live after the v1 fix)	High	Fixed 2026-07-23 (v2) in `engine/dd_wager.py` and ported to `app/index.html`: the comparison SE is now the PAIRED per-trial difference (candidate vs. empirical best), which respects CRN; and the band is a multiplicity-adjusted Bonferroni value `z* = Phi^{-1}(1 - alpha/k)` over the `k` comparisons-with-the-best (Hsu, 1996). Verified stable across seeds and neighbouring confidences on a flat scenario ($0) AND that it still recommends a positive wager on a clear-signal scenario ($1,200). See `docs/reviews/statistical-review-2026-07-23-v2.md`.	Resolved — Python and JS fixed; 33 tests passing incl. a paired-SE <= independent-SE invariant
11	The Daily Double engine's reported `win_probability` is the in-sample maximum over candidate wagers, so it is an upward-biased ("winner's curse") estimate of the true best win probability	Medium	Low	Documented in the engine docstring, the returned notes, and the v2 review. The *recommended wager* is well insulated (smallest wager on a multiplicity-controlled plateau, not the raw argmax); only the reported percentage is optimistic.	Accepted, documented — a debiased (data-split) estimate is the future fix
12	Tier 2 Final Jeopardy enumeration treated each opponent's correctness as independent, but real FJ correctness is positively correlated across players (shared clue difficulty)	Medium	Medium	Fixed in 2.0.0: Tier 2 and the Daily Double rollouts model right/wrong outcomes with a one-factor Gaussian copula at a correlation CALIBRATED to modern-era play since 2.1.0 (ADR-0004; published value in 2.0.0, ADR-0003), across all players including the user. The `correlation` parameter allows sensitivity runs.	Resolved (2.0.0) — the binary-to-latent conversion is approximate away from even accuracy, tracked under risk #5

#	Risk	Likelihood	Impact	Mitigation	Status
13	The Tier 2 enumeration is described as "exact"; it is exact in the sense of no sampling error, but is computed in IEEE-754 double precision, not exact rational arithmetic	Low	Low	Accumulated floating error (~1e-16) is far below the 1e-9 comparison tolerance, so no result is affected; white-paper wording tightened to "exact enumeration in double precision (no sampling error)".	Accepted, documented
14	Two assumptions about the real game may be wrong. (a) Equal tiebreaker odds: the engines give each tied player a 1/k share, but buzzer skill decides the tiebreaker clue, so a stronger player wins it more often. (b) Stale behavior data: second place's wager mix and the shape of third place's bets still come from games played from roughly the mid-1990s to 2012; contestants may wager differently now. The engines do not model change over time.	Medium	Medium	(b) mitigated in 2.1.0 for most parameters: accuracy, the outcome correlation, the leader's cover share and Daily Double behavior are calibrated to Seasons 38-41 (2021-2025, ADR-0004). Second place's mix has no modern source and is unchanged. Both remaining items are stated in `engine/behavior.py` and ADR-0003/ADR-0004. The tie rule itself is current (since 24 Nov 2014). Win percentages are presented as model outputs. Future fix: a modern source for second place's mix, and a tiebreaker-odds parameter.	Partly mitigated (2.1.0) — (a) open; (b) open for second place only
15	J! Archive terms of use. The modern-era calibration (ADR-0004) used J! Archive season statistics, and its terms forbid republishing data collected or derived from it. The calibrated values are visible in `engine/behavior.py` and the app's public source, and a strict reading could treat rounded model settings as derived data. A takedown request is possible.	Low	Medium	No statistic, table or quote from J! Archive appears in any document, commit or slide; documents name the constants and point to `engine/behavior.py`. The source notes stay in `data/private/`, which `.gitignore` excludes. The pages were read by hand, not scraped. On a takedown request, the calibrated values are removable (revert to the PUBLISHED Tesauro et al. values in one configuration block).	Accepted by Will, 2026-10-13